

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Problem Solving Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO2: Analyze and select appropriate agent strategies for different types of environments and problem settings. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

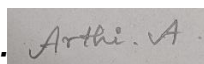
No. of Questions: 5

Annexure A Problem Solving Assignment

Code of Conduct:

I ARTHI A (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

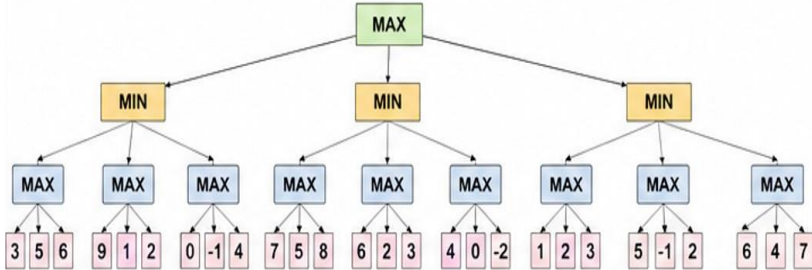
Signature of the Faculty

Total Marks Awarded

Answer All the Questions

Q.No	Application-Based Question	Major CO2 Concepts																																																
1	Smart Ambulance Navigation: An ambulance must travel from a hospital to an emergency location through a city road network. Some roads have higher travel costs, and an estimated distance to the destination is available for each junction. Design a search-based solution to identify the best route. Using the given road network and heuristic values, determine the route selected by your algorithm, show the sequence of junctions explored, calculate the relevant evaluation values, and justify whether the route is optimal.	Informed Search, A*, Heuristic Function																																																
2	Autonomous Warehouse Robot: A robot has to move from a charging station to a target shelf in a warehouse. The robot does not know all obstacles initially and discovers some obstacles only when it reaches nearby locations. Design an intelligent search strategy for the robot. Demonstrate the robot's decisions for the given grid, explain how it reacts when a new obstacle is discovered, and compare the approach with an offline search strategy. <table><tr><td>5</td><td>S</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr><tr><td>4</td><td>?</td><td>#</td><td>?</td><td>?</td><td>#</td><td>?</td><td>?</td></tr><tr><td>3</td><td>?</td><td>?</td><td>?</td><td>#</td><td>?</td><td>?</td><td>#</td></tr><tr><td>2</td><td>?</td><td>#</td><td>?</td><td>?</td><td>?</td><td>#</td><td>?</td></tr><tr><td>1</td><td>?</td><td>?</td><td>#</td><td>?</td><td>?</td><td>?</td><td>G</td></tr><tr><td></td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td></tr></table> S Charging Station (Start) G Target Shelf (Goal) # Obstacle (Unknown initially) ? Unknown cell	5	S	?	?	?	?	?	?	4	?	#	?	?	#	?	?	3	?	?	?	#	?	?	#	2	?	#	?	?	?	#	?	1	?	?	#	?	?	?	G		1	2	3	4	5	6	7	Online Search Agents, Informed Search
5	S	?	?	?	?	?	?																																											
4	?	#	?	?	#	?	?																																											
3	?	?	?	#	?	?	#																																											
2	?	#	?	?	?	#	?																																											
1	?	?	#	?	?	?	G																																											
	1	2	3	4	5	6	7																																											
3	University Examination Scheduling: A university needs to schedule six examinations (E1, E2, E3, E4, E5, and E6) into four available time slots (T1, T2, T3, and T4). Some examinations have common students and therefore cannot be conducted in the same time slot.	CSP, MRV, Forward Checking, Constraints																																																

	The following constraints must be satisfied: 1. E1 and E2 have common students. 2. E1 and E3 have common students. 3. E2 and E4 have common students. 4. E3 and E5 have common students. 5. E4 and E6 have common students. 6. E2 and E5 have common students. The university has three examination halls with the following capacities: <table><tr><th>Examination Hall</th><th>Capacity</th></tr><tr><td>Hall A</td><td>120 students</td></tr><tr><td>Hall B</td><td>90 students</td></tr><tr><td>Hall C</td><td>60 students</td></tr></table> <table><tr><th>Examination</th><th>E1</th><th>E2</th><th>E3</th><th>E4</th><th>E5</th><th>E6</th></tr><tr><td>Number of Students</td><td>100</td><td>80</td><td>70</td><td>60</td><td>90</td><td>50</td></tr></table> Tasks: a) Formulate the problem as a Constraint Satisfaction Problem (CSP) by clearly defining the variables, domains, and constraints. b) Use Backtracking with the Minimum Remaining Values (MRV) heuristic and Forward Checking to obtain a valid examination timetable. c) Show the domain of each variable after every assignment and provide the final timetable, ensuring that all student-conflict and hall-capacity constraints are satisfied.	Examination Hall	Capacity	Hall A	120 students	Hall B	90 students	Hall C	60 students	Examination	E1	E2	E3	E4	E5	E6	Number of Students	100	80	70	60	90	50	
Examination Hall	Capacity																							
Hall A	120 students																							
Hall B	90 students																							
Hall C	60 students																							
Examination	E1	E2	E3	E4	E5	E6																		
Number of Students	100	80	70	60	90	50																		
4	Game-Playing AI: You are developing an AI opponent for a board game. The current game position produces the following possible moves and terminal scores in the supplied game tree. Determine the move the AI should make by applying Minimax. Then apply Alpha-Beta Pruning and identify the branches that do not need to be evaluated. Explain how pruning improves the efficiency of the game-playing agent.	Game Playing, Minimax, Alpha-Beta Pruning																						

																																																		
5	Delivery Route Optimization: A logistics company wants to minimize the total distance travelled by a delivery vehicle visiting multiple customers. An initial route and several neighboring routes are provided. Design an optimization strategy to improve the route. Apply the strategy step-by-step to the given routes, determine the final route, explain whether it represents a global optimum, and suggest how the algorithm could escape a local optimum. Distance matrix (in km) <table><tr><th></th><th>D</th><th>A</th><th>B</th><th>C</th><th>E</th></tr><tr><th>D</th><td>-</td><td>6</td><td>9</td><td>8</td><td>7</td></tr><tr><th>A</th><td>6</td><td>-</td><td>3</td><td>5</td><td>6</td></tr><tr><th>B</th><td>9</td><td>3</td><td>-</td><td>4</td><td>5</td></tr><tr><th>C</th><td>8</td><td>5</td><td>4</td><td>-</td><td>3</td></tr><tr><th>E</th><td>7</td><td>6</td><td>5</td><td>3</td><td>-</td></tr></table> Some neighboring routes of the initial solution and their costs <table><tr><td>S1: D → A → C → B → E → D</td><td>= 26 km</td></tr><tr><td>S2: D → B → A → C → E → D</td><td>= 30 km</td></tr><tr><td>S3: D → A → B → E → C → D</td><td>= 24 km</td></tr><tr><td>S4: D → E → C → B → A → D</td><td>= 27 km</td></tr><tr><td>S5: D → A → E → B → C → D</td><td>= 23 km</td></tr><tr><td>S6: D → C → B → A → E → D</td><td>= 29 km</td></tr></table> From S5, the best neighbor has cost 22 km. From that solution, all neighbors have cost ≥ 22 km. Apply the chosen local search strategy and report the final route.		D	A	B	C	E	D	-	6	9	8	7	A	6	-	3	5	6	B	9	3	-	4	5	C	8	5	4	-	3	E	7	6	5	3	-	S1: D → A → C → B → E → D	= 26 km	S2: D → B → A → C → E → D	= 30 km	S3: D → A → B → E → C → D	= 24 km	S4: D → E → C → B → A → D	= 27 km	S5: D → A → E → B → C → D	= 23 km	S6: D → C → B → A → E → D	= 29 km	Local Search, Optimization, Hill Climbing
	D	A	B	C	E																																													
D	-	6	9	8	7																																													
A	6	-	3	5	6																																													
B	9	3	-	4	5																																													
C	8	5	4	-	3																																													
E	7	6	5	3	-																																													
S1: D → A → C → B → E → D	= 26 km																																																	
S2: D → B → A → C → E → D	= 30 km																																																	
S3: D → A → B → E → C → D	= 24 km																																																	
S4: D → E → C → B → A → D	= 27 km																																																	
S5: D → A → E → B → C → D	= 23 km																																																	
S6: D → C → B → A → E → D	= 29 km																																																	

QUESTION 1

Smart Ambulance Navigation

Major CO Concepts: Informed Search, A*, Heuristic Function

1. Problem Statement

An ambulance must travel from a hospital (source node S) to an emergency location (goal node G) through a weighted city road network. Each road has a travel cost, and every junction has an estimated remaining distance represented by a heuristic value $h(n)$. The objective is to select a route with minimum total travel cost. A* Search is selected because it combines the actual cost already travelled with an estimate of the remaining cost.

2. Problem Analysis

- Initial state: S, representing the hospital.
- Goal state: G, representing the emergency location.
- State: the current road junction occupied by the ambulance.
- Actions: move from the current junction to any directly connected junction.
- Path cost: sum of the travel costs of all roads traversed.
- Assumption: road costs are non-negative and the displayed graph represents the available road network.
- Objective: reach G with the least possible total travel cost.
- Constraint: the ambulance should not use a route whose total cost is higher when a cheaper goal route is available.

3. AI Technique Selection

A* Search is the most appropriate technique because this is a weighted shortest-path problem with useful heuristic information. Uniform-Cost Search considers only the cost already travelled, whereas Greedy Best-First Search considers only the estimated distance to the goal. A* combines both using $f(n)=g(n)+h(n)$, thereby balancing past cost and estimated future cost.

4. Justification of Algorithm

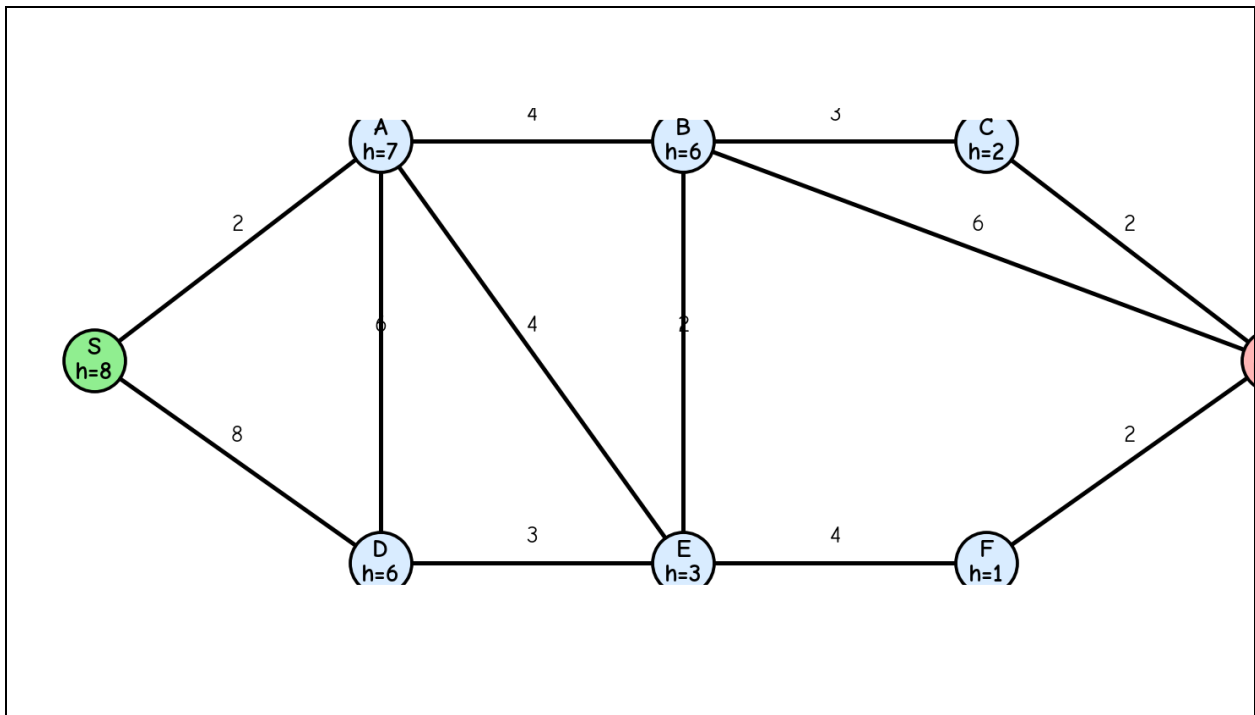
- A* is informed because it uses $h(n)$ to guide the search toward the emergency location.
- It is cost-sensitive because $g(n)$ represents the actual travel cost from S to n.
- If $h(n)$ is admissible, A* does not overestimate the remaining optimal cost and can guarantee an optimal solution.
- For an ambulance, minimizing travel cost is more important than merely reaching the goal quickly according to an approximate direction.

- A* usually explores substantially fewer irrelevant nodes than uninformed breadth-first search on a large weighted road network.

5. State Space Representation

The given graph contains the junctions S, A, B, C, D, E, F and G. The relevant road costs visible in the figure are: S–A=2, S–D=8, A–B=4, A–D=6, A–E=4, D–E=3, B–E=2, B–C=3, B–G=6, C–G=2, E–F=4, and F–G=2. The heuristic values are $h(S)=8$, $h(A)=7$, $h(B)=6$, $h(C)=2$, $h(D)=6$, $h(E)=3$, $h(F)=1$ and $h(G)=0$.

STATE-SPACE / ROAD-NETWORK DIAGRAM



6. Variables, Domains and Constraints

Not Applicable in the CSP sense. This is a path-search problem. The state variable is the current junction, the domain consists of its neighboring junctions, and the main constraint is that road traversal must follow an existing edge in the graph.

7. Heuristic Function

The heuristic function is the estimated remaining cost from node n to the goal G . In this problem the supplied $h(n)$ values are used directly:

Node	S	A	B	C	D	E	F	G
$h(n)$	8	7	6	2	6	3	1	0

An admissible heuristic never overestimates the actual minimum remaining cost to G . The provided values are intended to guide A*. For the optimality claim, the heuristic must be admissible (and, for the standard graph-search implementation, preferably consistent). Since $h(G)=0$, the goal has no estimated remaining cost.

8. Algorithm Design / Pseudocode

Pseudocode for A* Search:

A_STAR(start, goal):

OPEN $\leftarrow$ priority queue ordered by $f(n) = g(n) + h(n)$

CLOSED $\leftarrow$ empty set

$g(\text{start}) \leftarrow 0$

$\text{parent}(\text{start}) \leftarrow \text{NULL}$

insert start into OPEN

while OPEN is not empty:

$n \leftarrow$ remove node with smallest $f(n)$

 if $n = \text{goal}$:

 return reconstruct_path(parent, goal)

 if n is already in CLOSED:

 continue

 add n to CLOSED

 for each neighbor m of n :

 tentative_g $\leftarrow g(n) + \text{cost}(n, m)$

 if m is not in OPEN or tentative_g $< g(m)$:

$g(m) \leftarrow \text{tentative_g}$

$f(m) \leftarrow g(m) + h(m)$

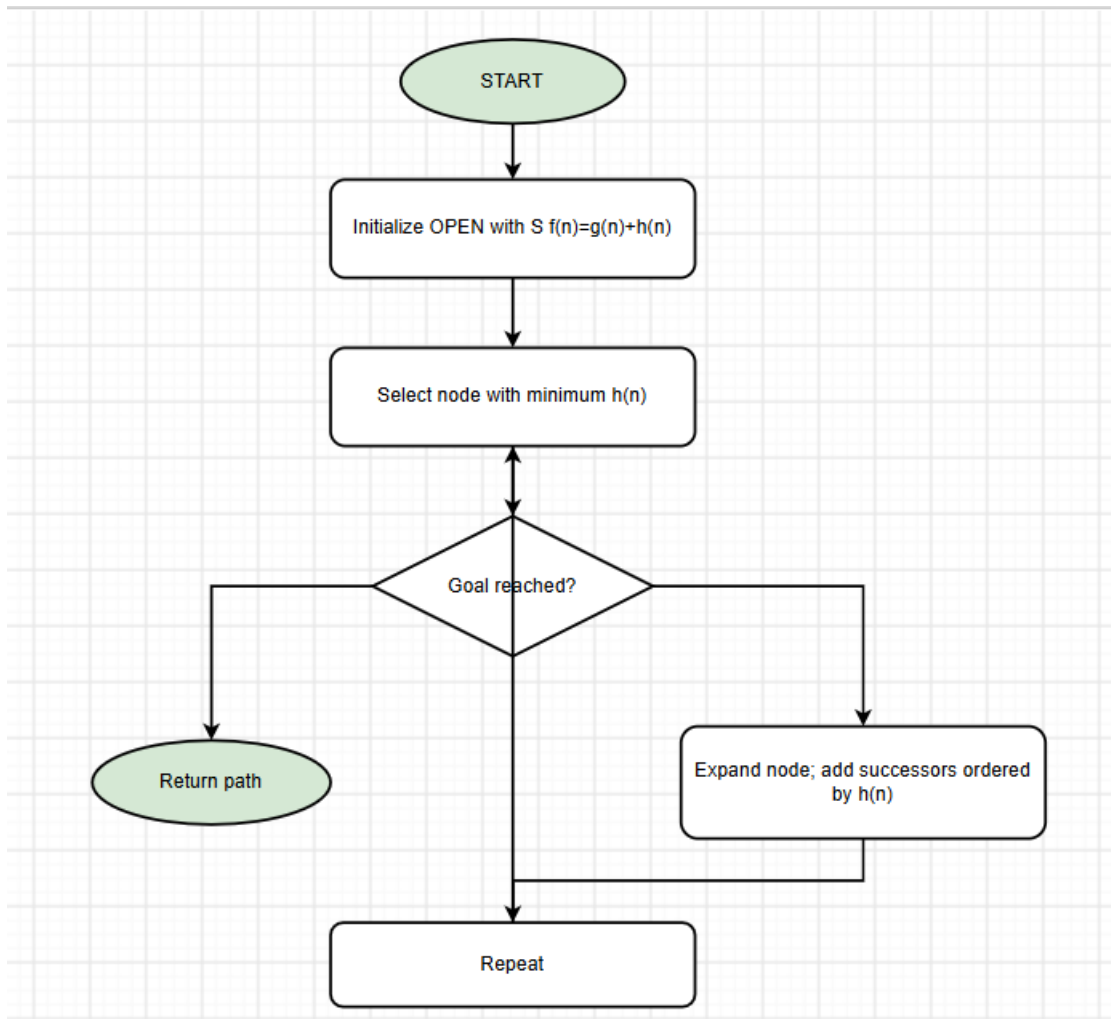
$\text{parent}(m) \leftarrow n$

 insert/update m in OPEN

return FAILURE

9. Flowchart / Algorithm Diagram

A* FLOWCHART – Draw.io



10. Working / Execution Trace

Starting from S, $g(S)=0$ and $f(S)=0+8=8$. Expand the node with the smallest f value at every step.

Expansion	Node expanded	$g(n)$	$h(n)$	$f(n)$	Important generated nodes
1	S	0	8	8	A: $g=2, f=9$; D: $g=8, f=14$
2	A	2	7	9	B: $g=6, f=12$; E: $g=6, f=9$; D remains $g=8, f=14$
3	E	6	3	9	F: $g=10, f=11$; D via E would be 9, so no improvement
4	F	10	1	11	G: $g=12, f=12$
5	G	12	0	12	Goal reached

Thus, one optimal route obtained from the displayed costs is $S \rightarrow A \rightarrow E \rightarrow F \rightarrow G$ with total cost $2+4+4+2 = 12$. Another route $S \rightarrow A \rightarrow B \rightarrow G$ also has cost $2+4+6 = 12$, so the graph contains two routes with the same optimal cost. Depending on tie-breaking in the priority queue, A^* may return either optimal route. The node expansion order under the stated tie-breaking is S, A, E, F, G. B may also appear in OPEN with $f=12$, so a different tie-breaking rule can change the order of equally promising nodes without changing the optimal cost.

11. Complexity Analysis

- Time complexity: in the worst case, A^* is exponential in the depth of the solution; commonly expressed as $O(b^d)$ for a general search tree, where b is branching factor and d is solution depth.
- Space complexity: $O(b^d)$ in the worst case because A^* stores frontier nodes and explored information.

12. Advantages and Limitations

- Advantages: uses both actual and estimated cost; can find an optimal path; effective for weighted road networks; easily incorporates domain knowledge through $h(n)$.
- Limitations: can consume significant memory; poor heuristics may make it behave closer to Uniform-Cost Search; performance decreases in very large graphs.
- If traffic conditions change continuously, a static A^* plan may become outdated and require replanning.

13. Conclusion

A^* is a strong choice for smart ambulance navigation because it explicitly balances the cost already incurred and the estimated remaining distance. For the given graph, the minimum route cost is 12, with $S-A-E-F-G$ being one optimal route and $S-A-B-G$ being another. An improved real-world system could update road costs using live traffic data and rerun A^* when congestion or road closures occur.

QUESTION 2

Autonomous Warehouse Robot

Major CO Concepts: Online Search Agents, Informed Search

1. Problem Statement

A warehouse robot must travel from the charging station S to the target shelf G. Initially, the robot does not know which cells contain obstacles. It discovers an obstacle only when the relevant location becomes observable. Therefore, a fixed offline path may become invalid during execution. The appropriate solution is an Online Search Agent that plans with the information currently available, executes an action, senses the environment, updates its internal map, and replans when necessary.

2. Problem Analysis

- Initial state: S at grid coordinate (1,5).
- Goal state: G at (7,1).
- Known/unknown environment: cells marked ? are initially unknown; obstacles marked # are obstacles in the supplied grid.
- Actions: move up, down, left or right when the move is legal.
- The robot receives new information after sensing nearby cells.
- Objective: reach G while avoiding newly discovered obstacles and minimizing unnecessary movement.
- Assumption: each legal grid move has unit cost and the robot can detect an obstacle before entering or immediately when it becomes relevant.

3. AI Technique Selection

An Online Search Agent is more suitable than a conventional offline A* search because the complete state space is not known initially. The robot cannot safely compute one permanent path from an unknown map. An online strategy can use A* or another informed planner on the currently known map, execute the next safe action, update the map, and replan.

4. Justification of Algorithm

- It works under partial environmental knowledge.
- It can react immediately when a new obstacle is discovered.
- It avoids committing to a route that depends on unknown cells remaining free.
- It combines sensing, planning, action and replanning in a continuous loop.
- Compared with offline A*, it is more robust to dynamic or previously hidden obstacles.

5. State Space Representation

The robot's state can be represented as (position, known-map). The position changes after each movement, while the known-map changes whenever the robot discovers a previously unknown cell. A practical implementation maintains two maps: an occupancy map and a set of unexplored/unknown cells.

WAREHOUSE GRID

5	S	?	?	?	?	?	?
4	?	#	2	?	#	?	?
3	?	?	?	#	?	?	#
2	?	#	?	1	2	#	?
1	?	?	#	?	6	?	G
	1	2	3	4	5	6	7

6. Variables, Domains and Constraints

Not Applicable as a CSP formulation. The relevant state variables are robot position and the robot's current belief about the map. The action domain is {Up, Down, Left, Right}. Constraints include grid boundaries, discovered obstacles, and safe movement.

7. Heuristic Function

For a four-directional grid with unit movement cost, Manhattan distance is appropriate:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

This heuristic estimates the minimum number of moves required when there are no additional obstacles. It is admissible for a static four-directional unit-cost grid because obstacles can only make the actual path longer, never shorter.

8. Algorithm Design / Pseudocode

ONLINE_A_STAR_ROBOT(start, goal):

 current $\leftarrow$ start

 known_map $\leftarrow$ initially known cells only

 while current $\neq$ goal:

 sense nearby cells

 update known_map

if the planned path is blocked or new information changes the route:

$\text{path} \leftarrow \text{A_STAR}(\text{current}, \text{goal}, \text{known_map})$

if no path exists in the current known map:

move to a safe unexplored cell / continue exploration

update map

else:

$\text{next} \leftarrow \text{first safe action on path}$

execute next

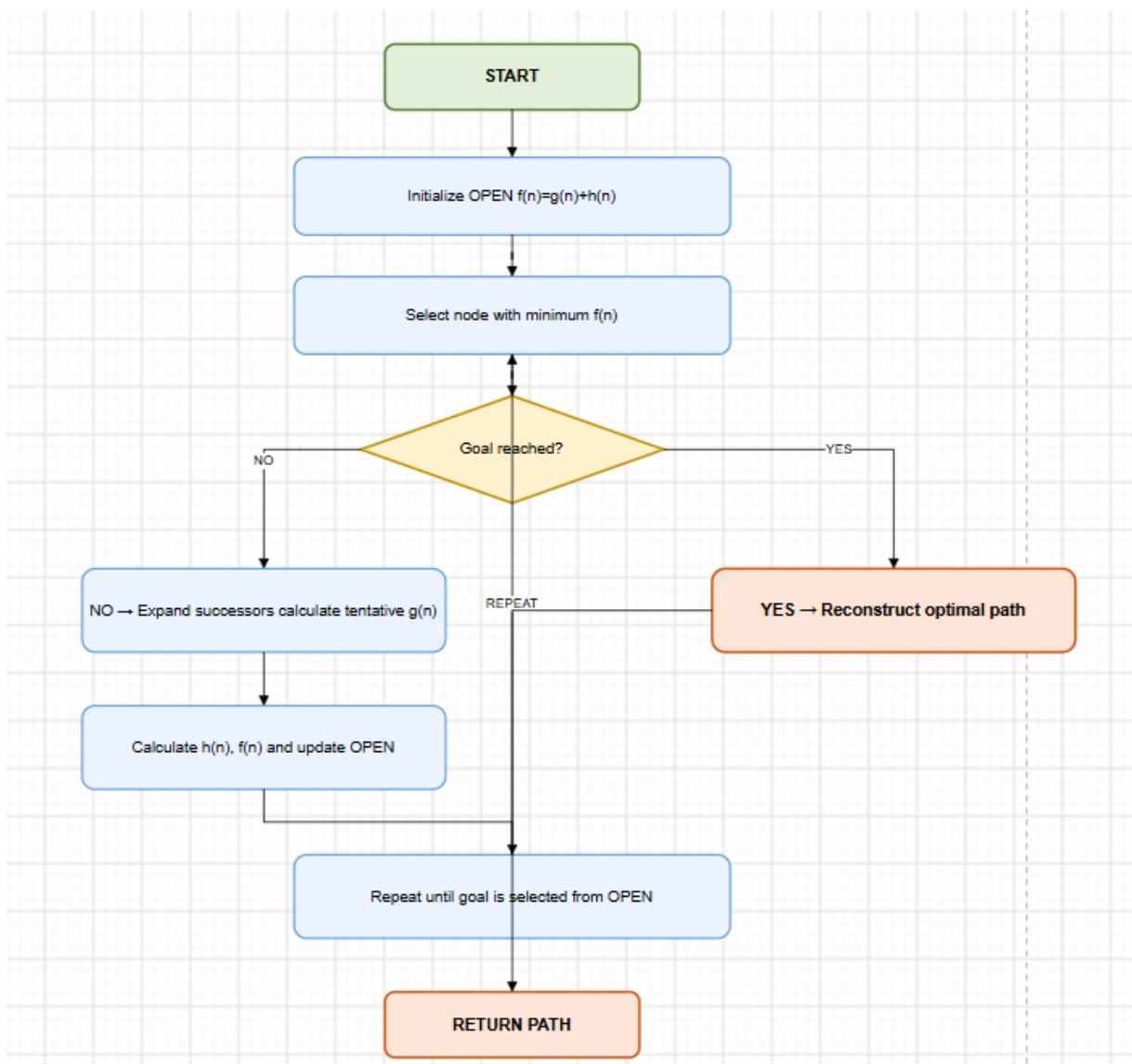
$\text{current} \leftarrow \text{next}$

sense again

return SUCCESS

9. Flowchart

ONLINE SEARCH AGENT FLOWCHART – Draw.io



10. Working / Execution Trace

A representative execution demonstrates why online replanning is required. The robot initially prefers a route that appears to move toward the goal. When it approaches a cell whose status was unknown, it senses the cell. If an obstacle is found, that route is removed from the known map and A* is run again.

Step	Robot decision	New information	Action
1	Start at S=(1,5)	Local cells are sensed; most cells remain unknown	Move toward the goal
2	Plan along a shortest-looking corridor	No blocking obstacle yet	Continue
3	At/near (5,5), inspect (5,4)	Obstacle # is discovered at (5,4)	Do not enter; replan
4	Replan around the blocked cell	Alternative cells through the right side are considered	Move toward (7,4)
5	Inspect (7,3)	Obstacle # is discovered	Do not enter; replan again
6	Use lower corridor through (6,3),(5,3),(5,2),(5,1)	Known obstacles are avoided	Continue
7	Reach G=(7,1)	Goal detected	Stop

One safe route after incorporating the displayed obstacles is: (1,5) → (2,5) → (3,5) → (4,5) → (5,5) → (6,5) → (7,5) → (7,4) → (6,4) → (6,3) → (5,3) → (5,2) → (5,1) → (6,1) → (7,1). The exact executed route can differ because the online agent depends on when each unknown obstacle becomes observable.

11. Complexity Analysis

- A* planning on an N-cell grid is worst-case exponential in search depth, although the actual grid implementation is usually much more manageable.
- Repeated replanning increases computation because A* may be executed multiple times.
- Space complexity is O(N) for the known map, OPEN/CLOSED structures and parent information.
- Online search trades additional planning effort for adaptability and safety in an uncertain environment.

12. Advantages and Limitations

- Advantages: adapts to new obstacles, suitable for partially known environments, supports informed planning, and reduces the risk of following an invalid fixed path.
- Limitations: repeated replanning can be computationally expensive; sensing errors can cause poor decisions; the robot may travel farther than an ideal offline route.

- Offline A* is faster when the complete map is known and static, but it is less robust when obstacles are discovered during execution.

13. Conclusion

An Online Search Agent provides the intelligence required for the warehouse scenario because the robot cannot assume that all unknown cells are free. By combining sensing, A*-based planning and replanning, the robot can continuously adapt to newly discovered obstacles. A practical extension is D* Lite or Lifelong Planning A*, which are designed for efficient repeated path planning in changing environments.

QUESTION 3

University Examination Scheduling

Major CO Concepts: CSP, MRV, Forward Checking, Constraints

1. Problem Statement

Six examinations E1–E6 must be scheduled into four time slots T1–T4. Some examinations share students and therefore cannot occur in the same time slot. In addition, each examination must be assigned to an examination hall whose capacity is sufficient for the number of students registered for that examination. The problem is formulated as a Constraint Satisfaction Problem (CSP), and Backtracking with the Minimum Remaining Values heuristic and Forward Checking is used to obtain a valid timetable.

2. Problem Analysis

- Examinations: E1, E2, E3, E4, E5, E6.
- Time-slot domain: {T1, T2, T3, T4}.
- Hall domain: {A, B, C}, with capacities A=120, B=90, C=60.
- Student counts: E1=100, E2=80, E3=70, E4=60, E5=90, E6=50.
- Conflict pairs: (E1, E2), (E1, E3), (E2, E4), (E3, E5), (E4, E6), (E2, E5).
- Exams in a conflict pair cannot share a time slot.
- A hall cannot accommodate an exam whose student count exceeds its capacity.
- For a practical timetable, a hall hosts at most one examination in a time slot.

3. AI Technique Selection

CSP is the natural representation because the problem consists of variables with finite domains and constraints between variable assignments. Backtracking provides systematic search, MRV chooses the most constrained variable first, and Forward Checking removes inconsistent values from the domains of unassigned variables immediately after each assignment.

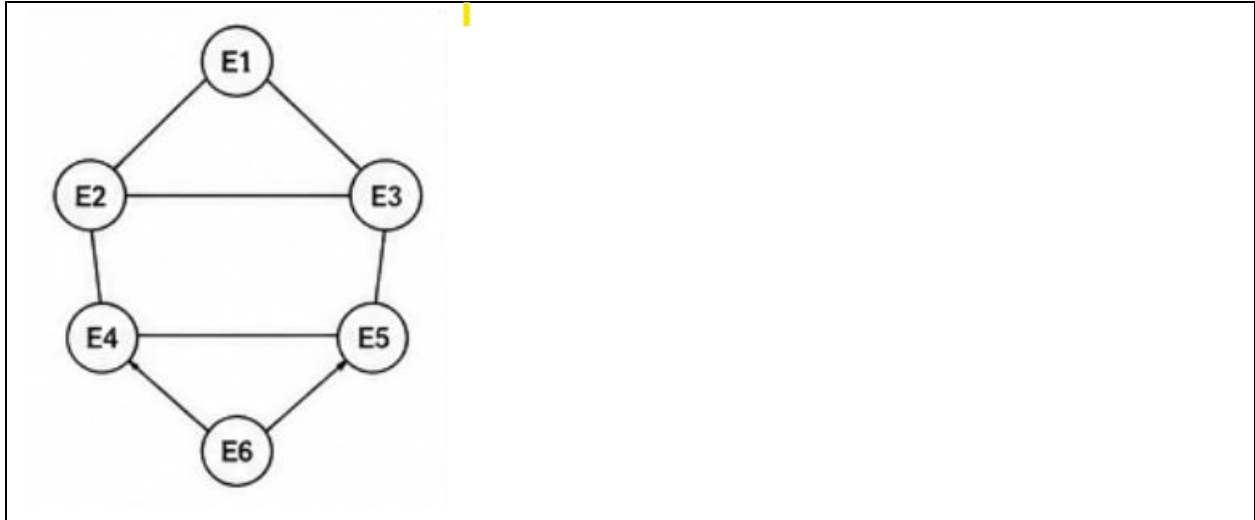
4. Justification of Algorithm

- CSP directly models examination scheduling as variable assignment under constraints.
- MRV reduces branching by selecting the variable with the fewest legal remaining values.
- Forward Checking detects future conflicts early instead of waiting until a complete timetable is built.
- The combination is more efficient than plain backtracking because many impossible partial assignments are rejected immediately.
- Hall capacity constraints are handled as unary constraints on hall domains.

5. State Space / Constraint Graph

Constraint graph vertices are E1–E6. An edge connects two exams that have common students. Therefore the graph edges are E1–E2, E1–E3, E2–E4, E3–E5, E4–E6 and E5–E6.

CONSTRAINT GRAPH – DRAW MANUALLY HERE



6. Variables, Domains and Constraints

Time-slot variables:

Variable	Initial domain
E1	{T1,T2,T3,T4}
E2	{T1,T2,T3,T4}
E3	{T1,T2,T3,T4}
E4	{T1,T2,T3,T4}
E5	{T1,T2,T3,T4}
E6	{T1,T2,T3,T4}

Hall domains based only on capacity:

Exam	Students	Valid halls
E1	100	{A,B}
E2	80	{A,B}
E3	70	{A,B}
E4	60	{A,B,C}
E5	90	{A,B}
E6	50	{A,B,C}

Binary constraints: $E1 \neq E2$, $E1 \neq E3$, $E2 \neq E4$, $E3 \neq E5$, $E4 \neq E6$ and $E2 \neq E5$ in the time-slot assignment. Hall constraints require the selected hall capacity to be at least the examination's student count.

7. Heuristic Function

MRV is the variable-ordering heuristic. For each unassigned exam, count the legal time slots remaining after current constraints are applied. Select the exam with the smallest remaining domain. If two exams have the same MRV value, a useful tie-breaker is the degree heuristic: select the exam involved in the largest number of constraints.

Forward Checking is the domain-filtering procedure. When E_i is assigned a slot, that slot is removed from every unassigned neighbor of E_i . If any neighbor's domain becomes empty, the assignment is rejected immediately and the algorithm backtracks.

8. Algorithm Design / Pseudocode

BACKTRACK(assignment, domains):

 if all examinations are assigned:

 return assignment

$X \leftarrow$ choose unassigned examination using MRV

 break ties using highest constraint degree

 for each value v in domain(X):

 if v is consistent with all assigned neighbors:

 assign $X \leftarrow v$

 new_domains $\leftarrow$ copy(domains)

 remove v from every neighbor's time-slot domain

 remove occupied halls from the hall domain for this slot

 check hall-capacity constraints

 if every unassigned variable has a non-empty domain:

 result $\leftarrow$ BACKTRACK(assignment, new_domains)

 if result $\neq$ FAILURE:

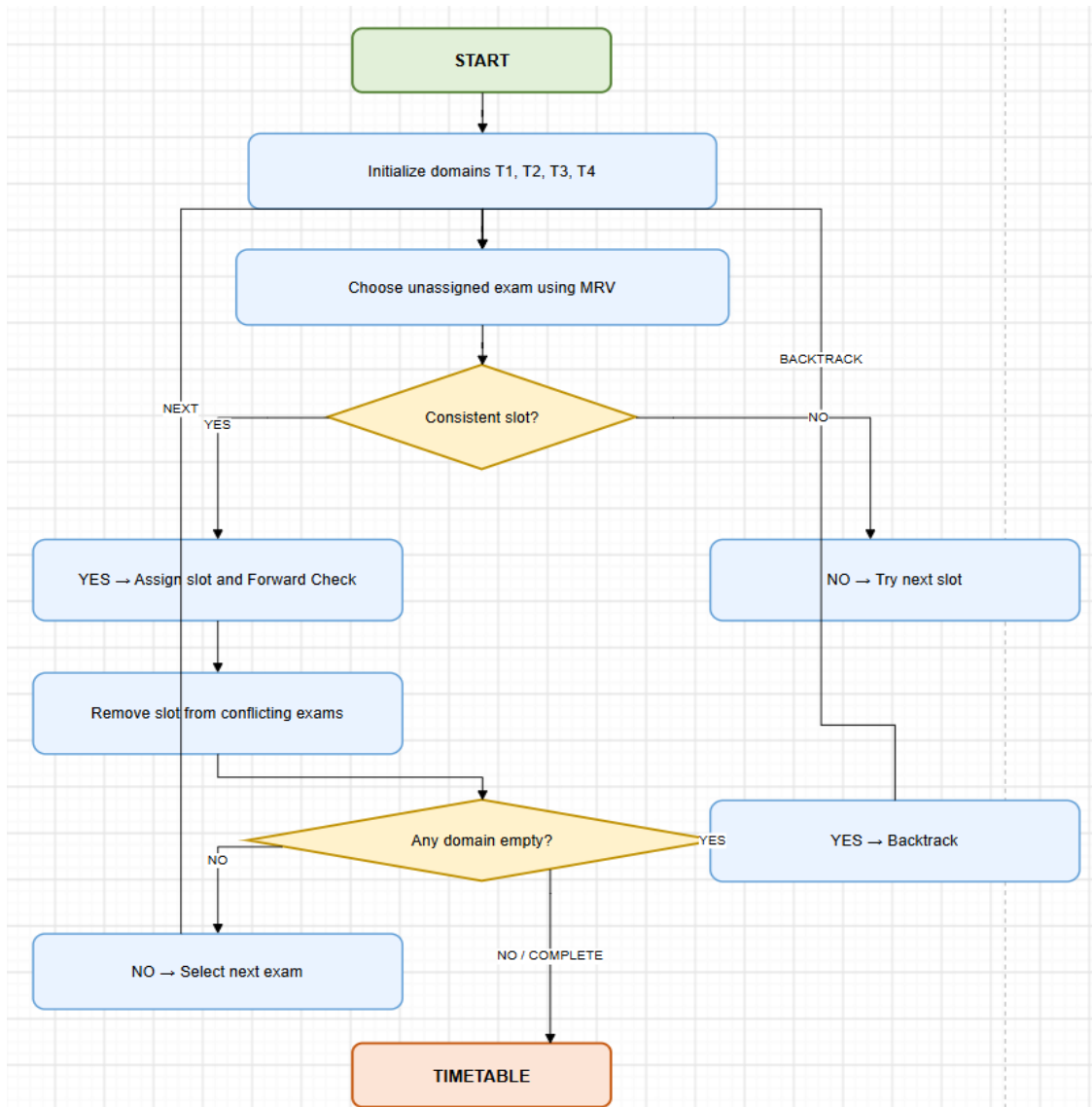
 return result

 undo assignment and restore domains

return FAILURE

9. Flowchart

CSP + MRV + FORWARD CHECKING FLOWCHART – Draw.io



10. Working / Execution Trace

All exams initially have four time slots. MRV therefore has a tie at the beginning; use the degree heuristic to select E2 because E2 participates in three conflict constraints (with E1, E4 and E5).

Step	Assignment	Remaining slot domains after Forward Checking
Initial	None	$E1:E2:E3:E4:E5:E6 = \{T1,T2,T3,T4\}$
1	$E2=T2$	$E1:\{T1,T3,T4\}; E4:\{T1,T3,T4\};$ $E5:\{T1,T3,T4\}; E3:\{T1,T2,T3,T4\};$ $E6:\{T1,T2,T3,T4\}$
2	$E1=T1$	$E3:\{T2,T3,T4\}; E2 \text{ fixed};$ $E4:\{T1,T3,T4\}; E5:\{T1,T3,T4\};$ $E6:\{T1,T2,T3,T4\}$
3	$E3=T2$	$E5:\{T1,T3,T4\}; E4:\{T1,T3,T4\};$ $E6:\{T1,T2,T3,T4\}$
4	$E4=T1$	$E6:\{T2,T3,T4\}; E5:\{T1,T3,T4\}$
5	$E5=T1$	No conflict because E5 is not adjacent to E1 or E4; E6 remains $\{T2,T3,T4\}$
6	$E6=T2$	All variables assigned; no domain is empty

A valid slot assignment is therefore: $E1=T1$, $E4=T1$, $E5=T1$ and $E2=T2$, $E3=T2$, $E6=T2$. Notice that the three examinations in each slot do not conflict with one another.

Hall assignment:

Time	Exam	Students	Hall	Capacity check
T1	E1	100	A	$100 \leq 120$
T1	E4	60	C	$60 \leq 60$
T1	E5	90	B	$90 \leq 90$
T2	E2	80	A	$80 \leq 120$
T2	E3	70	B	$70 \leq 90$
T2	E6	50	C	$50 \leq 60$
T3	—	—	—	Unused
T4	—	—	—	Unused

This schedule satisfies every student-conflict constraint and every hall-capacity constraint. Each hall is used by at most one exam in a slot.

11. Complexity Analysis

- Plain CSP backtracking has worst-case time $O(d^n)$, where n is the number of variables and d is the maximum domain size.
- Here $n=6$ and $d=4$ for the time-slot variables, so the naive search space has at most $4^6=4096$ slot assignments before pruning.

- MRV and Forward Checking reduce the practical search space by selecting constrained variables and removing inconsistent values early.
- Space complexity is $O(n \cdot d)$ for domains plus $O(n)$ recursion depth, excluding storage of the constraint graph.

12. Advantages and Limitations

- Advantages: systematic, complete for finite CSPs, detects conflicts early, and can incorporate additional constraints such as hall capacities.
- Limitations: worst-case exponential; performance depends on variable/value ordering; a very dense conflict graph can still create heavy backtracking.
- Further improvements include Degree heuristic, Least Constraining Value, Arc Consistency (AC-3), and optimization objectives such as minimizing the number of used slots.

13. Conclusion

The examination problem is naturally solved as a CSP. MRV and Forward Checking make the search substantially more efficient than naive backtracking. The final timetable uses two time slots and all three halls safely, while satisfying all six conflict constraints and all capacity restrictions.

QUESTION 4

Game-Playing AI

Major CO Concepts: Game Playing, Minimax, Alpha–Beta Pruning

1. Problem Statement

A board-game AI must select the best move for the MAX player from the supplied game tree. Terminal nodes contain utility scores. MAX chooses the largest value, while MIN chooses the smallest value. First, Minimax is used to evaluate the complete tree. Then Alpha–Beta Pruning is applied to avoid evaluating branches that cannot affect the final decision.

2. Problem Analysis

- Initial state: root node controlled by MAX.
- Second level: MIN nodes representing the opponent's choices.
- Third level: MAX nodes representing the next move of MAX.
- Terminal values: the leaf scores shown in the supplied game tree.
- MAX objective: maximize the final utility.
- MIN objective: minimize the final utility.
- Assumption: both players act optimally and the game tree is deterministic for the shown search depth.

3. AI Technique Selection

Minimax is appropriate for two-player, zero-sum, deterministic games where the opponent is assumed to play optimally. Alpha–Beta Pruning is then added as an optimization because it produces exactly the same minimax decision while eliminating branches that cannot influence the result.

4. Justification of Algorithm

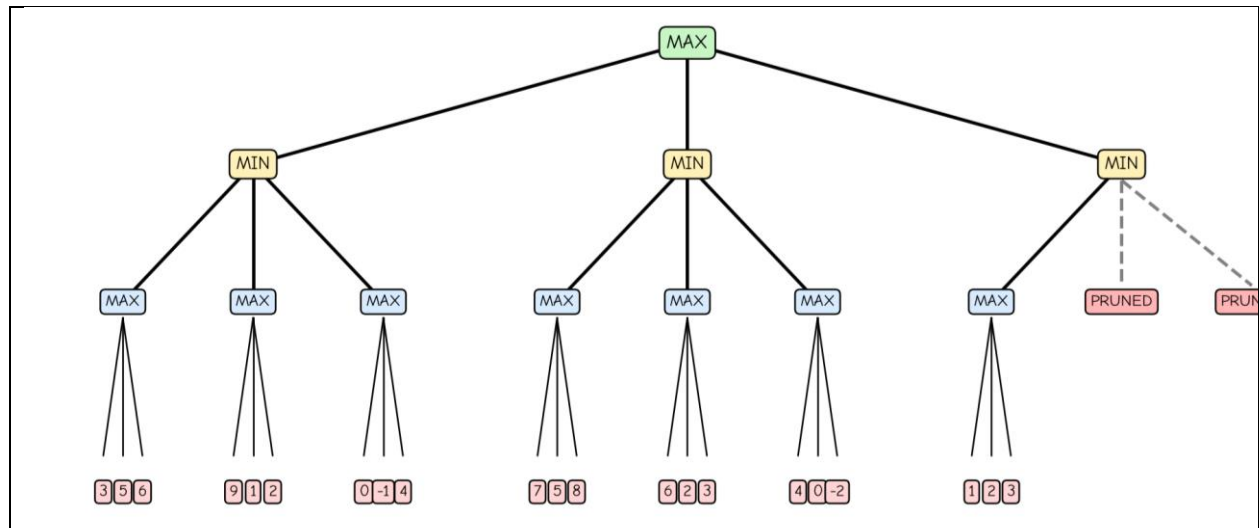
- Minimax explicitly models the opponent instead of assuming cooperative behavior.
- It evaluates terminal utilities and propagates them upward according to MAX/MIN rules.
- Alpha–Beta Pruning maintains lower and upper bounds, α and β , to identify irrelevant branches.
- Pruning improves efficiency without changing the final minimax value.
- Good move ordering can make Alpha–Beta dramatically faster than plain Minimax.

5. Game Tree Representation

The supplied tree has a MAX root, three MIN children, nine MAX nodes and 27 terminal leaves. The terminal groups from left to right are:

MIN branch	MAX child 1	MAX child 2	MAX child 3
Left	(3,5,6)	(9,1,2)	(0,-1,4)
Middle	(7,5,8)	(6,2,3)	(4,0,-2)
Right	(1,2,3)	(5,-1,2)	(6,4,7)

GAME TREE



6. Variables, Domains and Constraints

Not Applicable as a CSP. The game states form a tree. At MAX states, the selected value is the maximum child value; at MIN states, it is the minimum child value.

7. Heuristic Function

Not Applicable for the supplied full-depth tree because terminal utility values are directly available. In a deeper game, a depth-limited Minimax implementation would use an evaluation function at cutoff states to estimate the desirability of non-terminal positions.

8. Algorithm Design / Pseudocode

MINIMAX(node):

 if node is terminal:

 return utility(node)

 if node is a MAX node:

 value $\leftarrow -\infty$

 for each child:

 value $\leftarrow \max(\text{value}, \text{MINIMAX}(\text{child}))$

 return value

if node is a MIN node:

value $\leftarrow +\infty$

for each child:

value $\leftarrow \min(\text{value}, \text{MINIMAX}(\text{child}))$

return value

ALPHA_BETA(node, α , β , maximizing):

if node is terminal:

return utility(node)

if maximizing:

value $\leftarrow -\infty$

for each child:

value $\leftarrow \max(\text{value}, \text{ALPHA_BETA}(\text{child}, \alpha, \beta, \text{FALSE}))$

$\alpha \leftarrow \max(\alpha, \text{value})$

if $\alpha \geq \beta$:

break // prune remaining children

return value

else:

value $\leftarrow +\infty$

for each child:

value $\leftarrow \min(\text{value}, \text{ALPHA_BETA}(\text{child}, \alpha, \beta, \text{TRUE}))$

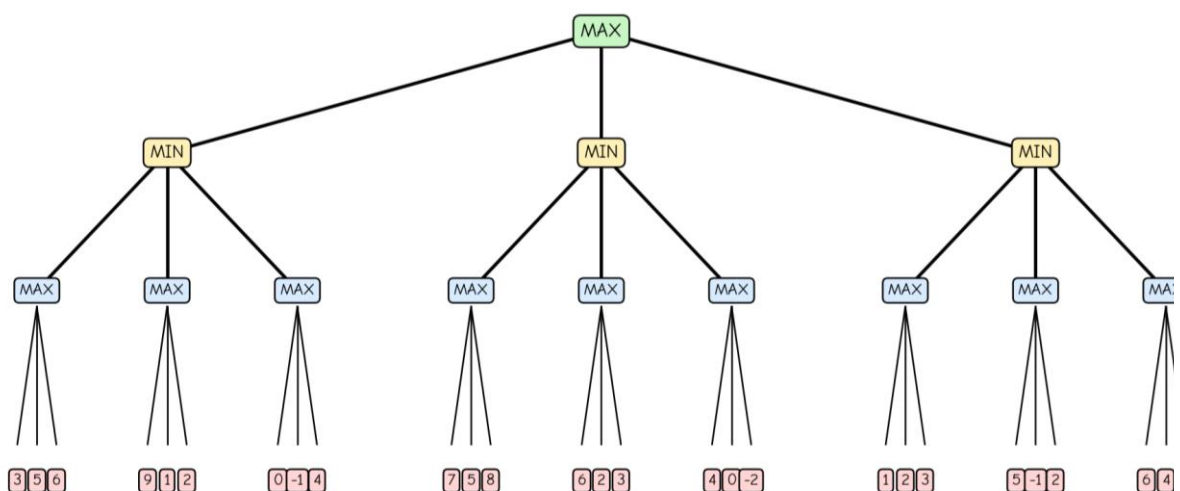
$\beta \leftarrow \min(\beta, \text{value})$

if $\alpha \geq \beta$:

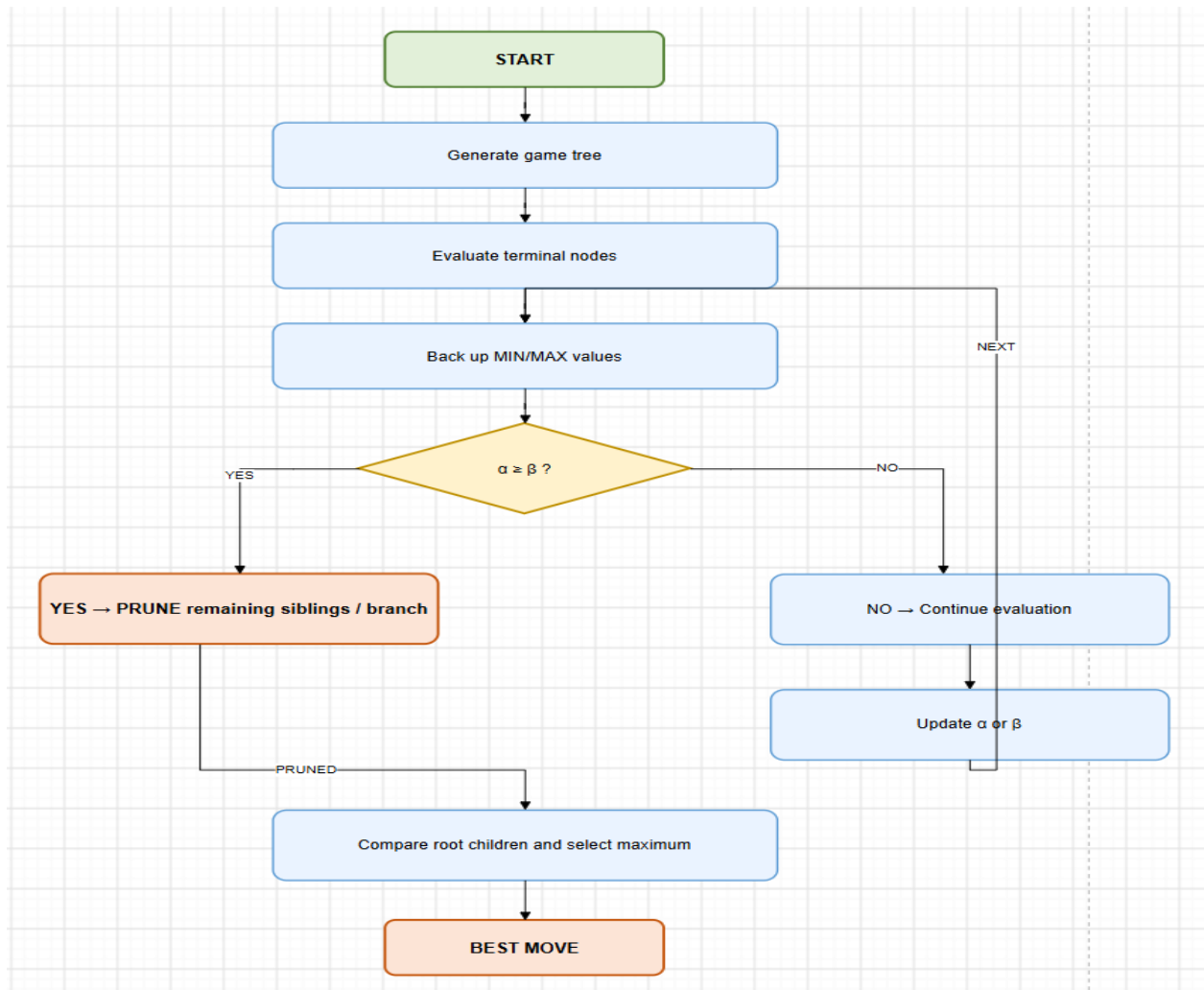
break // prune remaining children

return value

9. Flowchart / Algorithm Diagram



MINIMAX + ALPHA-BETA FLOWCHART



10. Working / Execution Trace

Minimax evaluation:

Root branch	MAX child values	MIN value
Left	6, 9, 4	4
Middle	8, 6, 4	4
Right	3, 5, 7	3

Therefore root MAX selects $\max(4,4,3)=4$. The best move is either the Left or Middle branch because both produce the same minimax value 4. If a deterministic tie-breaking rule chooses the first maximum, the Left branch is selected.

Alpha–Beta trace using left-to-right ordering:

Stage	α/β information	Result / pruning
Left MIN – first MAX	First MAX returns 6; β becomes 6	Continue
Left MIN – second MAX	First leaf is 9; local α reaches $9 \geq \beta=6$	Prune leaves 1 and 2
Left MIN – third MAX	Evaluates 0,-1,4; returns 4	Left MIN returns 4; root $\alpha=4$
Middle MIN – first MAX	Returns 8; $\beta=8$	Continue
Middle MIN – second MAX	Returns 6; $\beta=6$	Continue
Middle MIN – third MAX	Evaluates 4,0,-2; returns 4	Middle MIN returns 4
Right MIN – first MAX	Returns 3; β becomes 3	Since $\beta=3 \leq \text{root } \alpha=4$, prune remaining two MAX branches

Pruned terminal leaves: the two leaves (1,2) under the second MAX node of the left branch, plus all six leaves under the second and third MAX nodes of the right branch. Thus 8 of the 27 terminal values are not evaluated; 19 terminal values are evaluated.

The total tree contains 40 nodes (1 root + 3 MIN + 9 MAX + 27 leaves). With the above pruning, the two pruned MAX nodes on the right and one pruned MAX node on the left are not expanded, along with the associated terminal leaves.

11. Complexity Analysis

- Plain Minimax: $O(b^d)$ time and $O(bd)$ space with depth-first recursion, where b is branching factor and d is search depth.
- Alpha–Beta worst case: $O(b^d)$, when move ordering is poor.
- Alpha–Beta best case: approximately $O(b^{(d/2)})$, when the best moves are examined first.
- Space remains $O(bd)$ for depth-first Alpha–Beta.

12. Advantages and Limitations

- Advantages: optimal against an optimal opponent, logically simple, Alpha–Beta gives the same decision with fewer evaluations, and good move ordering greatly improves performance.
- Limitations: full-depth game trees become enormous; depth-limited search needs an evaluation function; poor move ordering reduces pruning.
- Transposition tables and iterative deepening can further improve practical game-playing performance.

13. Conclusion

Minimax gives a root value of 4, with the Left and Middle moves tied as optimal choices. Alpha–Beta Pruning reduces the number of terminal evaluations from 27 to 19 for the shown left-to-right ordering while preserving the exact minimax decision. Therefore, Alpha–Beta is an important efficiency improvement for game-playing AI.

QUESTION 5

Delivery Route Optimization

Major CO Concepts: Local Search, Optimization, Hill Climbing

1. Problem Statement

A logistics company wants to minimize the total distance travelled by a delivery vehicle that starts at depot D, visits customers A, B, C and E, and returns to D. Several candidate routes are supplied. The task is to use local search, specifically Hill Climbing, to improve an initial route by repeatedly replacing it with a better neighboring route.

2. Problem Analysis

- Initial solution: $S5 = D \rightarrow A \rightarrow E \rightarrow B \rightarrow C \rightarrow D$, with the printed cost 23 km.
- State: one complete delivery tour beginning and ending at D.
- Neighbor: a route obtained from the current route by a small permutation such as swapping two customers or applying a 2-opt reversal.
- Objective function: total travel distance.
- Goal: minimize the route cost.
- Constraint: every customer must be visited exactly once and the vehicle must return to D.

3. AI Technique Selection

Hill Climbing is appropriate because the problem is an optimization problem over a very large set of possible delivery tours. Instead of storing the entire search tree, Hill Climbing keeps only the current route and its neighbors. It is therefore memory-efficient and often finds good solutions quickly.

4. Justification of Algorithm

- The number of possible delivery tours grows factorially with the number of customers.
- Local search avoids maintaining a large frontier of complete paths.
- A route can be improved by simple neighborhood operations such as swap, insertion or 2-opt.
- For logistics, a near-optimal route found quickly can be more practical than exhaustive enumeration.
- The basic algorithm can be strengthened using random restarts, simulated annealing or tabu search to escape local optima.

5. State Space Representation

The supplied distance matrix is:

DELIVERY ROUTE

	D	A	B	C	E
D	0	6	9	8	7
A	6	0	3	5	6
B	9	3	0	4	5
C	8	5	4	0	3
E	7	6	5	3	0

6. Variables, Domains and Constraints

Not Applicable as a CSP. A solution is a permutation of the customers A, B, C and E between the fixed depot D at the beginning and end. The neighborhood is generated by local route modifications. Each customer must occur exactly once.

7. Heuristic / Evaluation Function

For minimization, the evaluation function is the total route distance:

$$\text{Cost}(R) = \sum \text{distance}(v_i, v_{(i+1)})$$

A lower cost is better. Unlike A*, Hill Climbing does not require an admissible heuristic estimating a remaining goal cost. The objective value itself is used to compare neighboring complete solutions.

8. Algorithm Design / Pseudocode

HILL_CLIMBING(initial_route):

 current $\leftarrow$ initial_route

 repeat:

 neighbors $\leftarrow$ generate_neighbors(current)

 best $\leftarrow$ neighbor with minimum route cost

 if cost(best) $\geq$ cost(current):

 return current

 current $\leftarrow$ best

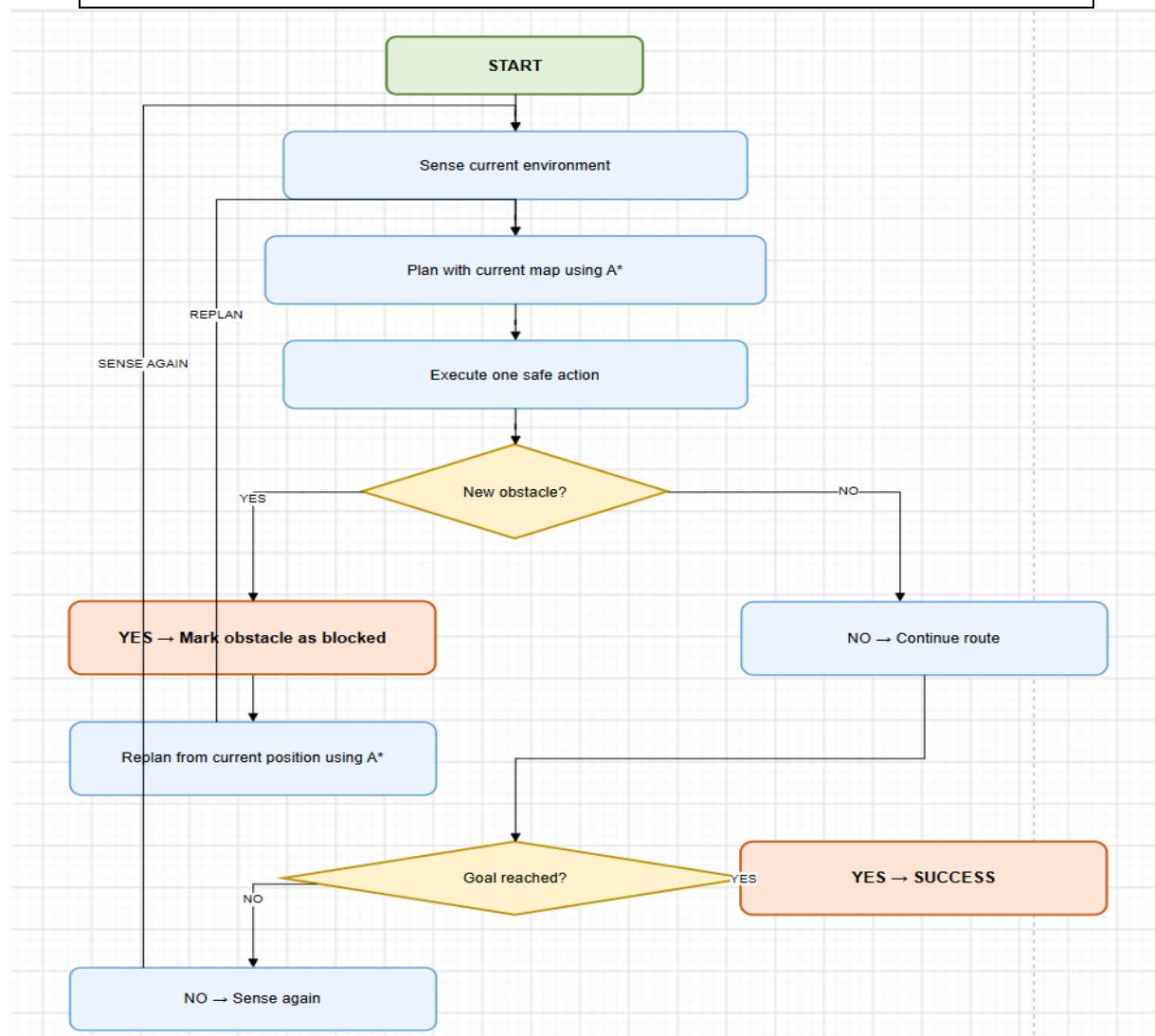
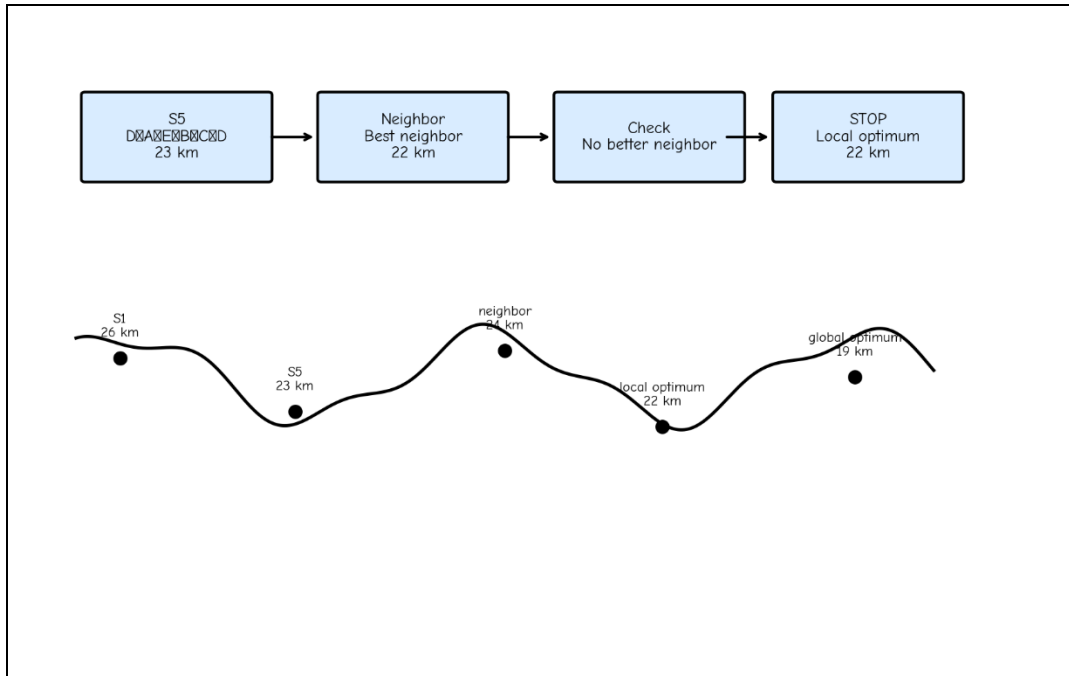
 until no improvement

 return current

For a stronger version, generate several random initial routes and run Hill Climbing from each. Return the best solution found. This is called Random-Restart Hill Climbing.

9. Flowchart / Algorithm Diagram

HILL CLIMBING FLOWCHART



10. Working / Execution Trace

The supplied candidate routes and their printed costs are:

Solution	Route	Printed cost
S1	D→A→C→B→E→D	26 km
S2	D→C→B→A→E→D	30 km
S3	D→A→B→E→C→D	24 km
S4	D→E→C→B→A→D	27 km
S5	D→A→E→B→C→D	23 km
S6	D→C→B→A→E→D	29 km

The figure states that S5 has a neighboring solution with cost 22 km and that all neighbors of that new solution have cost at least 22 km. Following the figure's intended Hill-Climbing trace:

Iteration	Current route/state	Cost	Decision
0	S5 D→A→E→B→C→D	23 km	Generate neighbors
1	Best displayed neighbor of S5	22 km	Move to the 22 km neighbor
2	Neighbors of the 22 km solution	≥ 22 km	No strictly better neighbor
3	Final	22 km (as stated in figure)	Stop at local optimum

Important consistency note for submission: the printed distance matrix and the printed 22 km value in the supplied figure do not appear to agree. Using the matrix exactly as shown, the minimum Hamiltonian tour is 23 km (for example D→A→B→C→E→D, and its reverse), so a 22 km route cannot be obtained from that matrix. If your instructor expects the figure's stated 22 km trace, reproduce the 22 km value. If the numerical answer is checked directly from the matrix, report 23 km and mention this discrepancy.

11. Complexity Analysis

- If k neighbors are generated at each iteration and the algorithm performs t iterations, the approximate time is $O(t \cdot k)$ plus the cost of evaluating each route.
- For swap-based neighborhoods, k can be $O(n^2)$ for n customers.
- Space complexity is $O(k \cdot n)$ if all neighbors are stored simultaneously, or $O(n)$ if neighbors are generated and evaluated one at a time.
- Compared with exhaustive TSP enumeration, local search is much more memory-efficient but does not inherently guarantee a global optimum.

12. Advantages and Limitations

- Advantages: simple, fast, low memory, suitable for large optimization spaces, and easy to adapt to different neighborhood operators.
- Limitations: can become trapped at a local optimum, plateau or ridge; solution quality depends on the initial route and neighborhood definition.
- Escape strategies: random restart, sideways moves, simulated annealing, tabu search, genetic algorithms, or larger neighborhood moves such as 2-opt/3-opt.

13. Conclusion

Hill Climbing provides an efficient local optimization method for delivery-route improvement. Starting from S5, the supplied figure indicates an improvement from 23 km to a 22 km local optimum. However, the displayed distance matrix mathematically gives a best tour of 23 km, so the assessment should acknowledge the apparent data inconsistency. In a real logistics system, Random-Restart Hill Climbing or Simulated Annealing would be preferable when avoiding local optima is important.

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (Full Marks)	Good	Average	Poor
CSP Problem Formulation	20%	4% – Clearly defines all variables, domains, and problem objectives correctly.	3% – Defines most variables and domains with minor errors.	2% – Partial formulation with some missing components.	0–1% – Incorrect or incomplete formulation.
Constraint Identification & Representation	20%	4% – Accurately identifies all student-conflict and hall-capacity constraints and represents them correctly.	3% – Identifies most constraints with minor errors.	2% – Identifies some constraints but misses important relationships.	0–1% – Constraints are largely incorrect or missing.
Application of Backtracking, MRV & Forward Checking	30%	6% – Correctly applies Backtracking, MRV, and Forward Checking with clear and logical steps.	4.5% – Applies the techniques correctly with minor mistakes.	3% – Partial application with unclear or inconsistent steps.	0–1.5% – Incorrect or inappropriate application of CSP techniques.
Step-by-Step Execution & Domain Updates	20%	4% – Provides a complete, logical execution trace and correctly updates domains after every assignment.	3% – Mostly correct execution with minor gaps.	2% – Provides limited steps with some incorrect domain updates.	0–1% – Incomplete or incorrect execution.

Final Timetable, Accuracy & Justification	10%	2% – Produces a valid conflict-free timetable satisfying all constraints and capacity requirements with clear justification.	1.5% – Valid timetable with minor errors in justification.	1% – Partially valid timetable with some constraint violations.	0–0.5% – Invalid or missing timetable.
--	-----	--	--	--	---